

Digital input

Knapp, buzzer, logik — och Arduinons första sinnesorgan.

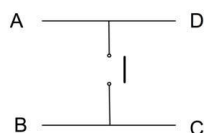
Vad du lärde dig idag

Den tredje träffen var ett paradigmskifte. Fram till nu har era program kört rakt fram: `setup`, `loop`, `digitalWrite`, `delay`. Ingen del av koden har reagerat på omvärlden. I dag fick programmen **läsa av** en knapp — och göra olika saker beroende på om den var tryckt eller inte.

Kvällens nyheter:

- `digitalRead(pin)` läser av en pinne och returnerar `HIGH` eller `LOW`.
- `pinMode(pin, INPUT_PULLUP)` aktiverar en intern pullup-resistor i Arduinon, så du slipper koppla en egen. Sido-effekt: knappens logik blir **inverterad** — tryckt knapp läses som `LOW`, släppt som `HIGH`.
- **if och else** låter programmet välja olika väg beroende på ett villkor. Operatorerna `==`, `!=`, `<`, `>` jämför värden.
- **Edge-detection** är mönstret för att agera **en gång** på en knapptryckning, även om knappen hålls nere i hundra loop-varv. Nyckeln: spara förra värdet, jämför med nuvarande, agera bara när de skiljer sig åt på rätt sätt.
- **Active buzzer** är plug-and-play — `digitalWrite(buzzerPin, HIGH)` ger ljud, `LOW` ger tyst. Ingen `tone()`, ingen frekvens, ingen resistor. Men: **klisterlappen stannar på**.

Ni byggde två kretsar parallellt (knapp + buzzer), kopplade ihop dem i en enda sketch, och såg Arduinons första sense-act-loop: den läser något från omvärlden, bestämmer sig för något, och agerar därefter. Det är i princip allt embedded-programmering handlar om.



Tactile switch — fyra ben, men elektriskt två par.
Tryckning kortsluter paren.



Active buzzer — svart cylinder med klisterlapp och plusmarkering på ovansidan. Lappen **stannar kvar**.

Repetition av de viktigaste begreppen

VARIABLER SOM KAN ÄNDRAS

I Modul 1 mötte ni `const int` — namngivna värden som aldrig ändras. I dag kom den första `int` utan `const`, och den första `bool`:

```
const int knappPin = 9;    // ändras aldrig
int tryckCount = 0;        // kan öka
bool larmPaslaget = false; // kan togglas
```

Regeln är enkel: om värdet **ska kunna ändras under tiden programmet kör**, släpp `const`. Om det är bestämt från början (ett pin-nummer, en tröskel, en maxtid), behåll `const` — det gör koden lättare att läsa och hindrar dig från att råka ändra värdet av misstag.

Datatyper ni sett hittills:

<code>int</code>	Ett heltal. Kan vara positivt, negativt eller noll. På Uno: -32768 till 32767.
------------------	--

<code>const int</code>	Heltal som inte ändras efter deklaration.
------------------------	---

<code>bool</code>	<code>true</code> eller <code>false</code> . Används för till/från-tillstånd.
-------------------	---

<code>byte</code>	Ett heltal 0–255. Bra för PWM-värden och byte-fält.
-------------------	---

En djupare genomgång av datatyper, scope och operatorer finns i Bilaga A.

PINMODE(PIN, INPUT_PULLUP) — MAGIN FÖRKLARAD

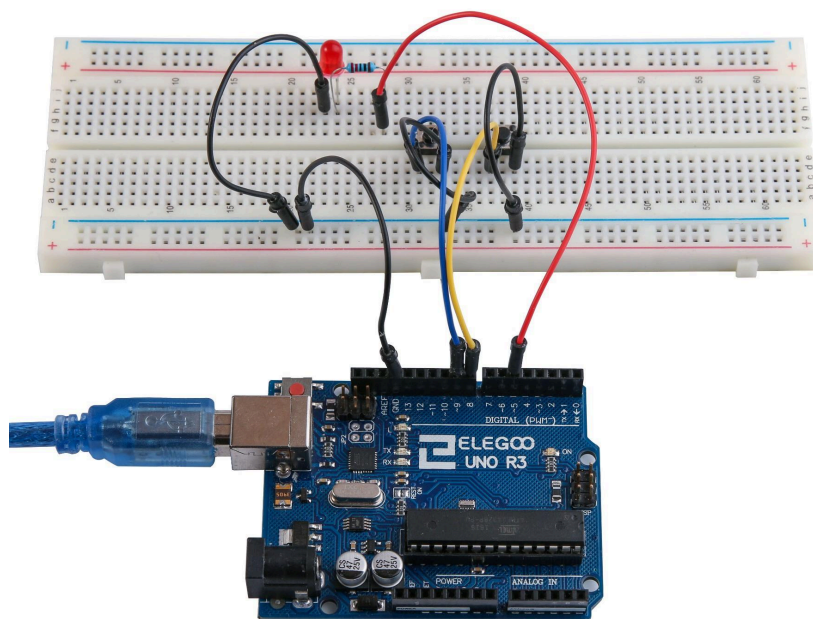
En Arduino-pinne som inte är kopplad till någonting **flyter**. Det betyder att dess spänning inte är bestämd — den kan vara något slumpmässigt mellan 0 och 5 V, ibland 2,4 V, ibland

0,1 V, ibland 4,7 V. `digitalRead` på en flytande pinne ger slumpmässiga resultat. Inte bra för en knapp.

Lösningen är en **pullup-resistor**: en resistor som kopplar pinnen till +5 V. När knappen är släppt, "drar" pullup:en pinnen till HIGH. När knappen trycks, kortsluter den pinnen till GND och drar den till LOW. Alltid ett definitivt värde, aldrig flytande.

`INPUT_PULLUP` säger åt Arduinon att aktivera en **intern** pullup-resistor — den finns inbyggd i chippet. Du behöver aldrig löda, aldrig koppla en extern resistor. En rad kod: `pinMode(knappPin, INPUT_PULLUP);`.

Konsekvensen är den inverterade logiken. **Tryckt knapp = LOW, släppt knapp = HIGH**. Svårt att vänja sig vid. Varje gång ni läser koden, tänk högt: "LOW betyder tryckt, LOW betyder tryckt".



*Knappkretsen på breadboard — två tactile switches och en LED som output. I kursen använder vi bara **en** knapp (A → D9) och Arduinons inbyggda LED på pin 13.*

BRYGGA TILL MODUL 4

`INPUT_PULLUP` är i själva verket en intern **pullup-resistor** — en "osynlig" resistor till +5 V som gör att en släppt knapp blir stabilt HIGH. I Modul 4 kommer du bygga en **riklig spänningsdelare** själv, med två resistorer, för att läsa av en fotocell. Skillnaden: i

Modul 3 får du bara två digitala lägen (HIGH/LOW). I Modul 4 får du en mellanspänning som kan variera och som `analogRead` kan mäta.

IF / ELSE

Syntaxen är:

```
if (villkor) {
  // kör det här om villkoret är sant
} else {
  // kör det här om det är falskt
}
```

Villkoret är ett uttryck som är antingen sant eller falskt. De vanligaste jämförelserna:

`a == b` **lika med**. Glöm inte det andra likhetstecknet — `=` är tilldelning, `==` är jämförelse.

`a != b` **olika**

`a < b` **mindre än**

`a > b` **större än**

`a <= b` mindre eller lika

`a >= b` större eller lika

Den vanligaste nybörjarbuggen är att skriva `if (x = 5)` när man menar `if (x == 5)`. Kompilatorn gnäller inte — det är giltig C++ — men programmet gör inte det du väntar dig. Två likhetstecken.

EDGE-DETECTION: AGERA PÅ FLANKEN

Problemet: en knapp som hålls ner är `LOW` i hundratals loop-varv innan användaren släpper den. Om du gör något vid **varje** `LOW`, händer det hundratals gånger.

Lösningen: reagera bara på **övergången** från HIGH till LOW — **flanken**. Spara förra värdet, jämför, agera bara när de skiljer sig åt:

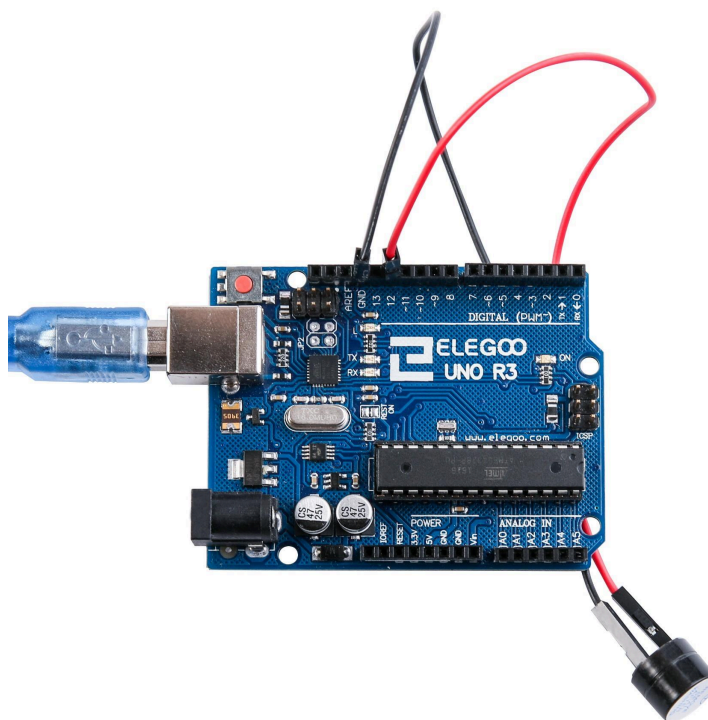
```
const int knappPin = 9;
bool larmPaslaget = false;
int lastState = HIGH;

void loop() {
  int state = digitalRead(knappPin);
  if (state == LOW && lastState == HIGH) {
    // flanken just nu – agera en gång
    larmPaslaget = !larmPaslaget;
  }
  lastState = state;
}
```

```
    delay(10); // liten paus mot studs
}
```

`!alarmPaslaget` är **logisk inversion** — utropstecknet betyder ”det motsatta”. Om `alarmPaslaget` var `false`, blir det `true`. Om det var `true`, blir det `false`. Det är så vi **togglar** ett tillstånd med en knapp.

`delay(10)` är en enkel **debounce**. Knappens metallblad studsar fysiskt några millisekunder när de möts, vilket ger flera falska flanker i rad. Tio millisekunder räcker för att släta över det i den här kursen. I produktion används fler tekniker — se Bilaga A.



Active buzzer direkt i Arduino-headers — långa benet (+) till D12, korta till GND. Ingen breadboard, ingen resistor.

ACTIVE BUZZER — INTE PASSIVE

I Elegoo-kittet finns **två** buzzrar: en **passive** (oftast blå och låg) och en **active** (svart, hög, med klisterlapp på ovansidan). Kursen använder den **aktiva**.

Active buzzer	Inbyggd oscillator. <code>digitalWrite(pin, HIGH)</code> → pip. <code>LOW</code> → tyst. En fast frekvens.
Passive buzzer	Ingen oscillator. Kräver <code>tone(pin, hz)</code> för att ge ljud. Kan spela olika tonhöjder.

Vi använder den aktiva för dess enkelhet. Men: **dra inte av klisterlappen**. Den är en fabriksdämpare från tillverkningsprocessen. Tekniskt fungerar buzzern utan, men den blir obehagligt hög. Lappen sitter på.

Bygg från minnet

Skriv från scratch en sketch som gör följande: När knappen på pin 9 är tryckt, piper buzzern på pin 12. När den släpps, är det tyst. Ingen edge-detection behövs för det här — en enkel `if/else` räcker.

SKELETT

```
const int knappPin = 9;
const int buzzerPin = 12;

void setup() {
  pinMode(knappPin, INPUT_PULLUP);
  pinMode(buzzerPin, OUTPUT);
}

void loop() {
  if (digitalRead(knappPin) == LOW) {
    digitalWrite(buzzerPin, HIGH);
  } else {
    digitalWrite(buzzerPin, LOW);
  }
}
```

Ladda upp. Tryck. Det piper. Du har byggt en ”första reaktiva krets” från minnet.

Hemma-övningar

ÖVNING 1 — TOGGLA LED MED EDGE-DETECTION

Använd Arduinons inbyggda LED (pin 13, `LED_BUILTIN`). När du trycker på knappen ska LED:en togгла — tänds om den är släckt, släcks om den är tänd. Detta kräver edge-detection-mönstret från ”Reagera på flanken”-sliden.

TESTA DIG FRAM

Börja med `bool ledPa = false;` och ett `lastState`-värde. I loopen: läs knappen, jämför med `lastState`. På flanken (`LOW && lastState == HIGH`), togгла `ledPa` och skriv `digitalWrite(LED_BUILTIN, ledPa ? HIGH : LOW)`. Den där sista raden är en kort `if/else`-syntax, läs den som ”om `ledPa` är true, skriv HIGH, annars LOW”.

ÖVNING 2 — RÄKNA KNAPPTRYCK OCH SKRIV UT

Skriv en sketch som räknar antalet knapptryckningar (med edge-detection) och skriver ut det totala antalet på Serial Monitor varje gång någon trycker. Du behöver lägga till `Serial.begin(9600);` i setup och `Serial.println(tryckCount);` i rätt del av loopen.

Nyckelinsikten: `Serial.println` ska INTE ligga i `loop()` utanför `if`-satsen. Då printar den 10 000 gånger per sekund. Den ska köras **bara** när en ny tryckning har registrerats — alltså inne i det lilla blocket där flanken detekteras.

ÖVNING 3 — KNAPP + BUZZER + MORSE

Skriv ett program där du kan morsa med knappen och buzzern piper direkt. Snabbt tryck = kort pip. Långt tryck = långt pip. Detta är faktiskt lättare än det låter — följ knappens tillstånd direkt: `if` knappen är `LOW` → buzzer `HIGH`, annars `LOW`. Ingen edge-detection behövs.

För extra credit: lägg till Arduinos inbyggda LED som ”visuell bekräftelse” så det tänds samtidigt som buzzern.

Vanliga fel och snabblösningar

Knappen gör ingenting	Glömt <code>pinMode(pin, INPUT_PULLUP)</code> . Utan den flyter pinnen. Eller: kabeln sitter i fel Arduino-pin.
Knappen ”sitter fast” vid tryck	Du läste av <code>HIGH</code> som ”tryckt”. Kom ihåg: med <code>INPUT_PULLUP</code> är LOW = tryckt.
Larmet växlar av/på random när jag håller knappen	Du saknar edge-detection. Agera på FLANKEN, inte på tillståndet.
Buzzern tjuiter konstant	Glömt <code>pinMode(buzzerPin, OUTPUT)</code> . Eller: du har skrivit <code>HIGH</code> utanför en <code>if/else</code> som aldrig vänder det tillbaka till <code>LOW</code> .
Serial Monitor skriver ut miljoner rader	Du printar direkt i loop utan <code>if</code> -skydd. Flytta <code>Serial.println</code> inuti <code>if</code> -blocket där det bara kör vid en ny händelse.
Knappen ger flera tryck åt gången	Studs. Lägg <code>delay(10);</code> på slutet av loopen — enklaste formen av <code>debounce</code> .

Snabbreferens

<code>pinMode(p, INPUT_PULLUP)</code>	Sätt upp som ingång med intern pullup.
<code>digitalRead(p)</code>	Returnerar <code>HIGH</code> eller <code>LOW</code> .
<code>if (x == y) { ... } else { ... }</code>	Villkorssats. Två likhetstecken för jämförelse.
<code>&& !</code>	Logiska operatorer: och, eller, inte.
<code>!x</code>	Logisk inversion. <code>!true = false</code> , <code>!false = true</code> .
Active buzzer	<code>digitalWrite(pin, HIGH)</code> = pip. Pin 12 i kursen. Klisterlappen stannar på.
Edge-detection-idiom	Spara <code>lastState</code> , jämför med nuvarande, agera bara på övergång.

Inför nästa träff

Modul 4 är den sista innan hackathonen. Tre nyheter står på schemat: **analogRead** (att läsa ett tal, inte bara HIGH/LOW), **spänningsdelaren** (tekniken som låter Arduinon mäta motstånd), och **Serial Monitor på allvar** (Arduinons verktygslåda för felsökning).

Ni får också två nya komponenter: en **fotocell** (en resistor vars motstånd sjunker i ljus) och en **tilt-sensor** (en metallkula som kortsluter två ben när den lutas). Den sista är vår anti-stöld-trigger i det slutliga larmet.

Glöm inte dator eller Elegoo-kitet hemma!